Batch Deduplication and Insertion-Time Prevention with a Shared Vector Canopy Index

Denis Robert
Independent Researcher

Abstract

Entity resolution is commonly deployed in two different workloads: an *in-place* batch operation that deduplicates an already-populated database, and an *insertion* operation that stops new duplicates before they land. These are usually treated as separate problems with separate machinery. This paper argues they should share a single vector index, and shows how. Dense row embeddings and a vector index serve, first, as the *cheap phase* of canopy-style blocking for batch deduplication: k-means (or any vector-native clustering) forms overlapping canopies, and Fellegi–Sunter (Splink) is the *expensive phase* that scores the candidate pairs within each canopy and joins them into connected components. The same index and the same canonical record mapping are then reused at insertion time: an incoming record is embedded, blocked to its canopies, and resolved against the canonical (deduplicated) store so that an existing record is rejected and a new record is admitted. We prototype this two-phase batch deduplication on a synthetic Canadian person database that genuinely contains duplicates, measure cluster-level precision/recall/F1 under varied canopy overlap, and demonstrate that the canonical index correctly rejects a re-insert and accepts a new record. We find that embedding-recoverability — whether a true duplicate is placed near its mate by the embedding model — is the binding constraint, and that canopy overlap is the primary recall lever. We close with the calibration caveat from the companion Calibration Paradox work: candidate pairs drawn from resemblance-selected neighbourhoods are even more biased than value-equality collisions, so any prior fitted over them must be re-anchored and the threshold re-tuned jointly.

1 Introduction

Entity resolution determines whether two records refer to the same real-world entity. In a persistent database, two distinct workloads demand resolution:

1. **Batch (in-place).** The database already exists and contains duplicates — accumulated over time, imported from multiple sources, or merged from acquisitions. The task is to find all duplicate groups and collapse them, producing a canonical, deduplicated database.

2. **Insertion.** New records arrive continuously. The task is to prevent future duplicates before they are stored — resolve each incoming record against the current database and reject or merge it if it matches.

These are conventionally addressed as separate problems: batch dedup as an offline blocking-and-linkage job, insertion as a query-time retrieval-and-resolve step. The central claim of this paper is that *both workloads naturally share one artifact*: a dense vector index over row embeddings of the database. The index provides the cheap candidate generation for batch dedup, and, once the database is canonicalised, it is the same index that insertion-time blocking queries. This yields three concrete benefits:

- **One embedding cost.** Each record is embedded once; neither workload re-embeds the population. The repository's persistence design already reloads an index without re-embedding [1].
- **One structural primitive.** The "canopy" that the cheap phase produces for batch dedup is the same shape as the top-k neighbourhood an insertion query retrieves: a set of resemblance-selected candidates that the expensive Fellegi–Sunter stage then scores.
- **One canonical subject.** After batch dedup, the index is rebuilt to contain canonical cluster representatives; insertion-time blocking then queries a duplicate-free store, so the duplicates that batch dedup removed cannot be re-created.

This paper is organised as follows. Section 2 distinguishes the three principal entity-resolution workflows — deduplication, cross-database linking, and merging — and locates the shared-index idea within them. Section 3 recalls the two-stage Fellegi–Sunter pipeline from the companion whitepaper [1]. Section 4 describes the canopy-style cheap phase — vector-native clustering with overlapping multi-assignment — and the expensive connected-components phase. Section 5 states the dual-use motivation that unifies batch dedup and insertion-time prevention on one index. Section 6 reports a prototype experiment on a synthetic duplicate-bearing database. Section 7 discusses the embedding-recoverability ceiling and the calibration caveat. Section 8 relates the approach to the literature, Section 9 concludes.

2 Use cases: deduplication, linking, and merging

Entity resolution is not a single task. It appears in at least three distinct workflows, and the differences among them matter for how a vector index is built, queried, and canonicalised.

1. **Deduplication (one database).** A single, already-populated database contains duplicate groups representing the same entity. The task is to find every such group and collapse it into one canonical record. This is the in-place batch workload of Sections 4–6: a single input population is both the candidate source and the canonical subject, and the output is a deduplicated database.
2. **Linking (two or more databases).** Two (or more) separately maintained databases hold records that may refer to the same entities, and the task is to establish the *correspondence* between them — "this record in database A is the same person as that record in database B" — *without* collapsing either database. The matched pairs are used to join analytics, to reconcile identifiers, or to propagate updates between systems, while each source keeps its own records. This is the `link` (or `link_only`) variant of the Fellegi–Sunter settings: two input datasets with distinct "source dataset" labels are compared against each other, and the output is a set of match edges, not collapsed clusters.
3. **Merging (collapse across databases).** The linking step is followed by a consolidation step: two databases are linked, then their matched records are merged into a single combined entity set, with conflicting fields resolved and duplicates eliminated. Merging therefore *reduces to deduplication*: once the cross-database correspondence is known (or once the sources are physically concatenated), the problem is again to deduplicate a single combined population and produce canonical records.

These three are related but not interchangeable. *Deduplication* and *merging* both end in canonical records, so they share the connected-components machinery; the only difference is whether

the input is one population (dedup) or several concatenated ones (merge). *Linking* is genuinely distinct: the output is a set of cross-database edges that *preserves* both source databases rather than collapsing them, so there is no single canonical index to maintain and no cluster to consolidate — only a persistent mapping between record identifiers across sources.

The shared-index thesis of this paper extends to linking with one adaptation. The cheap phase — vector clustering into overlapping resemblance canopies — is index-agnostic to whether the canopies span one database or several: the candidate pairs are simply those records (from any source) that share a canopy, and the expensive phase scores each pair under the same Fellegi–Sunter comparisons. For *linking*, the vector index and its canopies can be built over the concatenation of all sources (one shared embedding space), and the *link_only* settings — which attach a "source dataset" column and emit pairwise edges instead of connected-component clusters — are used in place of `dedupe_only`. For *merging*, the same link edges feed a consolidation pass that produces the combined deduplicated index. The dual-use property of Section 5 (one canonical index serving batch and insertion workloads) is specific to deduplication and merging, where a canonical subject exists; linking, by contrast, maintains a *correspondence*, and its "insertion-time" analogue is matching a newly arriving record in one source against the other source's index while still preserving both.

3 Two-stage Fellegi–Sunter pipeline

The companion whitepaper [1] describes a two-stage entity-resolution pipeline: dense row embeddings (sentence-transformers/all-MiniLM-L6-v2) persisted in a FAISS *IndexFlatIP* store, with each query embedded and blocked to its top-k cosine neighbours, after which Splink computes a match probability over explicit field comparisons (Jaro–Winkler for names and address, a date-of-birth comparison, an email comparison) and returns a match when the probability exceeds a threshold τ. The pipeline is designed for *insertion-time* resolution: given one query record, retrieve a small candidate set and decide. The present paper generalises the same two stages to the batch workload.

4 Canopy-style batch deduplication

4.1 Cheap phase: vector-native clustering as canopies

For batch deduplication the object is not a single query but the whole database. The cheap phase groups the stored vectors into candidate sets — *canopies*. The canonical recipe is canopy clustering [3]: a cheap distance measure places each record into overlapping candidates (a wide capture radius gathers members, a tighter criterion stops growth), and an expensive similarity is applied only within the resulting canopies. The overlap is deliberate: it prevents a true duplicate from being missed purely because it straddles a cluster boundary.

A vector index supplies a natural cheap phase. In the prototype we use FAISS k-means as the coarse quantizer and *multi-assignment* for overlap: each record is assigned to its top m centroids rather than a single one. With $m > 1$ the canopies overlap, and the candidate pairs are exactly the record pairs that share a centroid. This is a direct vector analogue of the two-threshold canopy procedure: the number of centroids C and the assignment depth m stand in for the capture and stop thresholds. Because vectors are L2-normalized and k-means operates on cosine-consistent geometry, the canopies reflect embedding similarity.

4.2 Expensive phase: Fellegi–Sunter over canopies + connected components

Each canopy is a shared block. Fellegi–Sunter (Splink) scores every candidate pair within a canopy under the field comparisons and a decision threshold τ. Pairs scoring at or above τ become match edges, and connected components over those edges yield the duplicate groups ("clusters"). This is batch deduplication as transitive closure: if record A matches B and B matches C, all three are placed in one canonical cluster even if A and C were never directly compared. Splink's clustering utilities compute exactly this on the predicted edges.

Implementation note: Splink's `dedupe_only` linker requires a single input table, so rather than passing a separate edge table we encode canopy membership as wide `anchor_`j columns (one per centroid assignment) on the node table and give Splink one blocking rule per column. This reconstructs exactly the canopy candidate pairs while satisfying the single-table constraint.

4.3 The candidate-pair cost surface

The cheap phase controls the cost of the expensive phase. With C canopies and assignment depth m, a record belongs to up to m canopies, and each canopy of size s yields $s(s-1)/2$ candidate pairs. The total candidate-pair budget therefore grows with cluster size ("quadratically in the number of records per canopy") and with m (through overlap). Raising C shrinks canopies and lowers the budget but risks missing cross-boundary duplicates; raising m recovers recall that hard partitions lose but inflates the budget. This precision–recall–cost trade-off of (C, m) is the batch analogue of the online k trade-off in the whitepaper [1].

5 Dual use: batch dedup and insertion-time prevention share one index

The pragmatic motivation for using a vector index as the cheap phase is that the same index serves both workloads:

One embedded index, two workloads. The batch pass runs offline over the populated index: cluster (cheap phase), run Fellegi–Sunter within canopies, collapse connected components into canonical records, and rebuild the index to hold one canonical representative per cluster. The insertion pipeline then queries that *same* canonical index: an incoming record is embedded, blocked to its top-k canopies, and resolved against canonical cluster representatives. The embedding cost is paid once; the persistence design in the repository [1] (`index.faiss` + metadata, reloaded without re-embedding) makes this reuse operationally natural.

Two consistency requirements make the claim non-trivial rather than automatic:

1. **The index must reflect the deduplicated database.** Raw batch dedup over the original index sees duplicated vectors as separate records. After collapsing duplicates, the index must be rebuilt from the canonical clusters (deduplicated vectors, remapped metadata) so insertion-time blocking queries a duplicate-free store — otherwise the index still "contains" the very duplicates it was meant to prevent. The store's `add`/`delete`/`reindex` semantics are the seam for this.

2. **Canonical representatives.** For insertion, the resolvable subject is the consolidated cluster (e.g. one representative per cluster), not every member of the original duplicate set. The batch pass must emit a vector-position → canonical-cluster mapping, which is exactly what insertion-time blocking consumes.

6 Prototype experiment

We implement `scripts/experiment_batch_dedup.py` in the companion repository, which prototypes the full loop (cheap clustering → Splink dedupe/connected components → canonicalisation → insertion check) on a synthetic Canadian person database that genuinely contains duplicate pairs.

6.1 Setup

A base population of N synthetic Canadian person records (Faker with a custom Canadian address provider, 30% independent address/email missingness) has a near-duplicate "twin" inserted for a 3% fraction, so the reference index genuinely contains matching records (the same construction as the duplicate-bearing benchmark of the whitepaper [1]). Two twin styles are used:

identical the twin is an exact copy of the base record — a positive control that trivially exercises the full canopy-dedupe machinery; and

varied the twin is a perturbed near-duplicate (typos, address/email variation), exposing whether the embedding itself recovers the pair.

The cheap phase runs FAISS k-means with C centroids and assignment depth m; Splink scores the within-canopy pairs at $\tau = 0.85$ (untrained default m/u, prior 10^{-4}); connected components yield cluster ids. Cluster-level precision/recall/F1 are measured against the known ground-truth duplicate pairs. In a validation environment (a 3,000-record base), the identical-twin control at $C = 128, m = 2$ reached recall 1.0 and F1 0.968 (precision 0.938), with 90/90 twins recovered by the cheap phase.

6.2 Overlap is the recall lever

Table 1 reports cheap-phase (canopy) twin recall against assignment depth m. Raising m recovers materially more true duplicates: at $m = 2$ only a fraction of varied twins share a canopy, whereas at $m = 8$ all 90 are captured. This is the direct prediction of Section 4: hard partitions lose cross-boundary duplicates, and overlap recovers them. It also confirms the central empirical claim of this paper — *embedding-recoverability is the binding constraint*: with the MiniLM embedding, exact twins are recovered by the embedding and the pipeline; varied twins are recovered only once aggressive overlap compensates.

Table 1: Cheap-phase (canopy) recall of the 90 base/twin duplicate pairs as a function of assignment depth m ($C = 128$ centroids).

m	identical twins recovered	varied twins recovered
2	90 / 90	6 / 90
8	90 / 90	90 / 90

The varied-twin result at $m = 8$ also carries forward to end-to-end linkage: cluster metrics were F1 0.90 (precision 0.90, recall 0.90) at $\tau = 0.85$, with 81 true positives and 9 false negatives among the 90 ground-truth duplicate pairs. The false negatives are exactly the cheap-phase misses that overlap at $m = 8$ still cannot recover for the noisy records. These are synthetic-sample measurements, not a general guarantee; they serve to demonstrate the mechanism and its data-dependent sensitivity, consistent with the tone of the companion work [1, 2].

6.3 Insertion-time reuse

After canonicalisation (one representative per cluster, twins removed, index rebuilt), the prototype resolves two incoming records against the canonical store: an identical re-insert of a base record, and a genuinely unrelated new record. The re-insert is retrieved and matched at match probability 1.0 against the canonical position of the original record and rejected as a duplicate; the unrelated record retrieves candidates but no match above threshold and is accepted as new. This demonstrates the shared-index loop: the same canonical index that batch-dedup built also serves insertion-time duplicate prevention.

7 Discussion and caveats

7.1 Embedding-recoverability is the ceiling

The dominant failure mode is not the linkage stage but the cheap phase: if the embedding does not place a true duplicate near its mate, no canopy overlap or threshold tuning can recover it. This mirrors the whitepaper's "blocking recall is the binding constraint" finding for the online pipeline [1]. In this prototype the MiniLM embedding recovers exact twins reliably but is sensitive to address edits for varied twins. The implication is that improvements to the cheap phase (richer embeddings, multi-view retrieval, or a dedicated canopy-aware index) are the lever that raises the batch-dedup ceiling, not more aggressive Splink settings.

How much this sensitivity matters depends on the data pipeline upstream of entity resolution. The operational problem that originally motivated this line of work supplies person records whose addresses are already standardized against an external postal reference engine before they reach either the embedding stage or the linkage stage, so a variant record's address is already in a canonical street/unit/city/postcode form rather than an arbitrary free-text form. Under that precondition the address edit that dominated the embedding drift in this synthetic measurement — where variation is injected into the raw address string — is largely removed, so the address-driven recoverability loss is not expected to recur in that setting. The findings here are therefore best read as a lower bound on what a *generic* embedding sees when fed un-standardised address text. In the general case — where addresses arrive un-parsed and inconsistent across sources — the measured sensitivity indicates that a better embedding strategy could pay off: either richer or domain-adapted encoders that weigh the more stable identity fields (name, birth date) ahead of the volatile address field, or a multi-view retrieval that embeds identity, contact, and address fields separately so that a changed address does not pull a true duplicate out of the canopy. The concrete choice remains data-dependent and should be evaluated with the same cheap-phase-recall protocol used here, on the actual address distribution of the deployment.

7.2 Field-position sensitivity of the embedding

A related question is whether the *ordering* of fields inside the serialized row affects the embedding's canopy behaviour — that is, whether fields earlier in the string carry more weight than later ones. We probed this with a dedicated script, `experiment_field_ordering.py`, using three measurements.

First, a *positional gradient*: a fixed-content probe sentence plus a single unique marker token moved to each position was embedded and compared with a no-marker baseline. The cosine distance of the marker is strongly front-loaded — 0.083 at position 0 versus roughly 0.012 mid-string — then plateaus, with only a small rise at the final position. Earlier fields are therefore more weighty, but the effect is confined to the first couple of tokens rather than increasing monotonically across

the whole string. This is consistent with the mean-pooled, positionally-embedded architecture of all-MiniLM-L6-v2.

Second, a *permutation-invariance* index: for records with all five fields present, the mean pairwise cosine across 24 field orderings of the same record is 0.973. Reordering thus moves the embedding by roughly 2.7% — a real but modest perturbation, smaller than the token-level gradient, because whole labeled fields saturate attention less than isolated tokens.

Third, the *canopy impact*: re-embedding the reference population under several orderings and clustering each with the same canopy parameters ($C = 128$, $m = 2$) changes cheap-phase recall of the varied twins in a measurable but bounded way. Leading with the long, discriminative *address* field recovered the most twins (0.978, 88/90) at the *lowest* candidate budget; the default order recovered 0.944 (85/90); and *identity-first* recovered the fewest (0.922, 83/90). Field ordering is therefore a small, mostly free lever on cheap-phase recall — but this result is data-dependent, here favouring the field that is most informative on this population, and it sits within the embedding-recoverability ceiling discussed above. As with any embedding choice, the winning order should be selected on deployment data with the same cheap-phase-recall protocol, not assumed generally.

7.3 Calibration caveat from the companion work

The companion Calibration Paradox paper [2] shows that refitting Fellegi–Sunter parameters on resemblance-selected candidate pairs inflates the fitted prior: candidate sets drawn from records that already resemble each other concentrate the non-match mass around near-collisions, so a free $\hat{\lambda}$ drifts upward and the classes overlap. Canopy-based batch dedup makes this *worse*, not better: embedding-neighbourhood candidate pairs are even more resemblance-biased than value-equality collisions, so a prior fitted over them is expected to drift beyond the 7.2×10^{-3} observed with value blocking. The practical rule from that paper applies a fortiori here: *anchor the prior* (default or blocking-adjusted), fit m/u with the prior pinned, and select τ jointly on a validation set.

7.4 Operational considerations

Batch dedup and insertion-time prevention have different latencies and cadences, and the shared index must be maintained coherently: deletes and updates from dedup must propagate to the canonical store before insertion-time blocking relies on it. In practice the operating loop is "deduplicate in batches, prevent at the boundary continuously," with periodic re-clustering or incremental index maintenance as the population evolves. This is operational scaffolding rather than an algorithmic limit, and it is the same seam identified in the whitepaper's discussion of index maintenance.

8 Related work

Canopy clustering was introduced by McCallum, Nigam, and Ungar for efficient blocking before expensive reference matching [3]; the cheap-then-expensive structure is exactly what a vector index plus Fellegi–Sunter reproduces. Embedding-based blocking has been used to build candidate sets for downstream matchers and training in DeepER [4], and similarity-based (including vector) blocking has been compared directly with exact-match blocking in the survey spirit of Zeakis et al. [5] and DeepBlocker [6]. Learned blocking functions that select candidate generation from labelled seeds (AutoBlock [7]) are data-driven analogues of canopy selection. The novel emphasis here is the *dual use*: the same embedded, canonicalised index serves both the batch-dedup cheap phase and insertion-time prevention, a property the blocking literature treats only implicitly as "the index

feeds the matcher." To the best of our knowledge the explicit two-workload sharing of one canonical vector index is under-emphasised in prior work.

9 Conclusion

A vector index unifies batch deduplication and insertion-time prevention. For batch dedup, the index supplies the canopy cheap phase (vector-native clustering with overlapping multi-assignment), and Fellegi–Sunter is the expensive phase that scores within-canopy candidate pairs and joins them into connected components. Once the database is canonicalised, the same index resolves incoming records to stop future duplicates. A prototype on a synthetic duplicate-bearing database demonstrates the full loop, confirms that overlap is the recall lever, and shows that the canonical index rejects re-inserts and accepts new records. The two caveats that govern practical use are the embedding-recoverability ceiling and the prior-inflation caveat from the companion Calibration Paradox work; together they dictate that the cheap phase, not the linkage stage, is where batch-dedup quality is won or lost, and that any fitted prior must be re-anchored and the threshold re-tuned jointly.

References

[1] Denis Robert. A two-stage entity resolution pipeline with FAISS blocking and Splink matching. Technical report, Independent Researcher, 2026.

[2] Denis Robert. The calibration paradox in Fellegi–Sunter processes. Technical report, Independent Researcher, 2026.

[3] Andrew McCallum, Kamal Nigam, and Lyle H. Ungar. Efficient clustering of high-dimensional data sets with application to reference matching. In Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pages 169–178, 2000.

[4] Muhammad Ebraheem, Saravanan Thirumuruganathan, Shafiq Joty, Mourad Ouzzani, and Nan Tang. Distributed representations of tuples for entity resolution. Proceedings of the VLDB Endowment, 11(11):1454–1467, 2018.

[5] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. Pre-trained embeddings for entity resolution: An experimental analysis. Proceedings of the VLDB Endowment, 16(9):2225–2238, 2023.

[6] Sanjib Das, AnHai Doan, Paul Suganthan G. C., Chaitanya Gokhale, Pradap Konda, Yash Govind, and Derek Paulsen. The case for blocking: A systematic study of blocking for entity resolution. Technical report, 2020. https://arxiv.org/abs/2004.02588.

[7] Wei Zhang, Hao Wei, Bunyamin Sisman, Xin Luna Dong, Christos Faloutsos, and David Page. AutoBlock: A hands-off blocking framework for entity matching. In Proceedings of the 13th ACM International Conference on Web Search and Data Mining, pages 744–752, 2020.

Use of Artificial Intelligence

During the preparation of this work the author used an AI coding and writing assistant (the DeepSeek V4 Flash 0731 large language model) to assist with drafting and editing the manuscript

and preparing the experiments and the accompanying prototype code. The tool was not listed as an author. The design and reported results were reviewed and verified by the author, who takes full responsibility for the content, including any remaining errors and/or omissions.